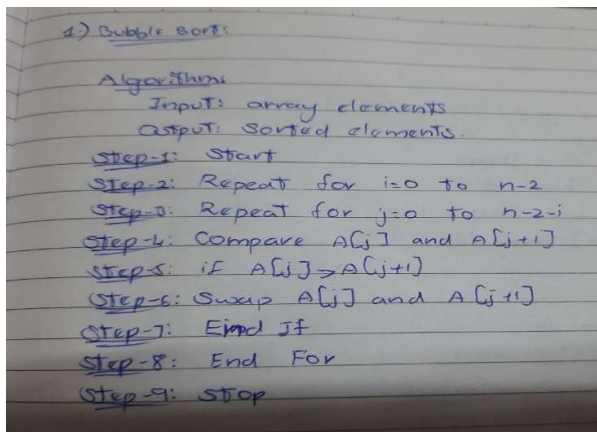


## PRACTICAL-01

**AIM:** Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

### 1.) BUBBLE SORT

#### ALGORITHM:



#### PROGRAM:

```
#include <iostream>

using namespace std;

class bubblesort {
public:
    int arr[100], size, i, j, temp;

    void getinput() {
        cout << "enter the size of the array:";

        cin >> size;

        cout << "enter the elements:";

        for (i = 0; i < size; i++) {
            cin >> arr[i];
        }
    }

    void printelements() {
```

```

        for (i = 0; i < size; i++) {
            cout << arr[i] << " ";
        }
    }

    void logic() {
        for (i = 0; i < size - 1; i++) {
            for (j = 0; j < size - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
};

int main() {
    bubblesort bubble;
    bubble.getinput();
    cout << "before sorting:";
    bubble.printelements();
    bubble.logic();
    cout << "sorted elements:";
    bubble.printelements();
    return 0;
}

```

### OUTPUT:

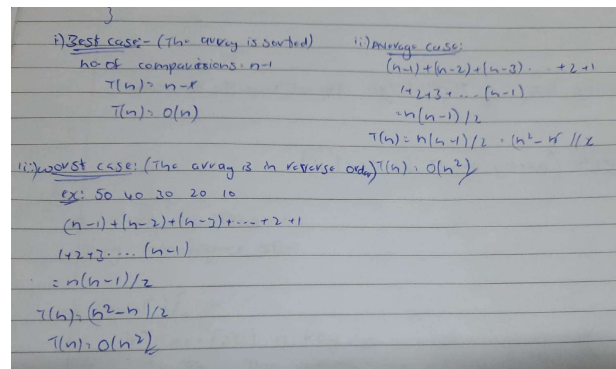
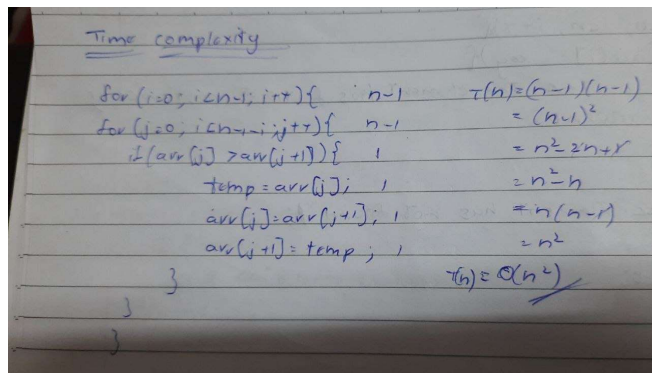
```

Output
enter the size of the array:5
enter the elements:5
9
7
1
3
before sorting:5 9 7 1 3 sorted elements:1 3 5 7 9

=== Code Execution Successful ===

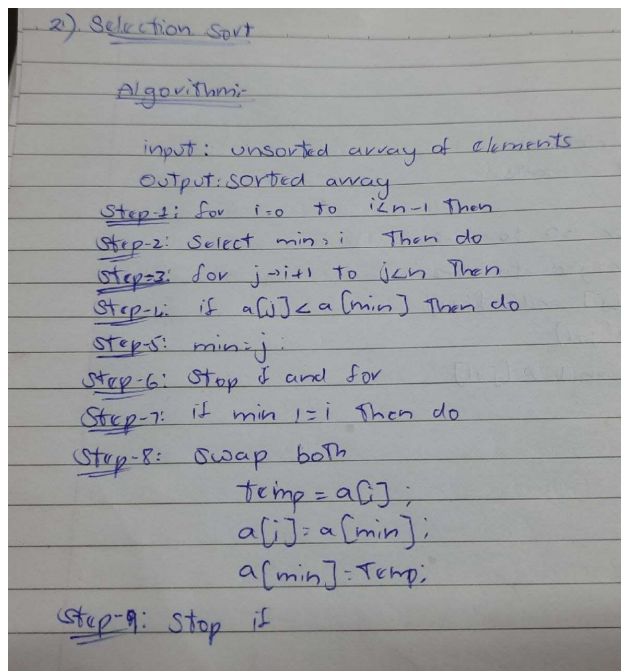
```

## TIME COMPLEXITY:



## 2.) SELECTION SORT:

### ALGORITHM:



### PROGRAM:

```
#include <iostream>
using namespace std;
class selectionsort{
public:
int arr[50], size, i, j, temp, min;
void getinput(){
cout<< "enter the size of the array:";
cin>> size;

cout<< "enter the elements:";
for(i = 0; i < size; i++){
cin >> arr[i];
}
}
```

```

void printelements(){
for(i = 0; i < size; i++){
cout << arr[i] << " ";
}
}

void logic(){
for(i = 0; i < size - 1; i++){
min = i;

for(j = i + 1; j < size; j++){
if(arr[j] < arr[min]){
min = j;
}
}

temp = arr[i];
arr[i] = arr[min];
arr[min] = temp;
}
}
};

int main(){

selectionsort selection;

selection.getinput();

cout << "before sorting:";
selection.printelements();

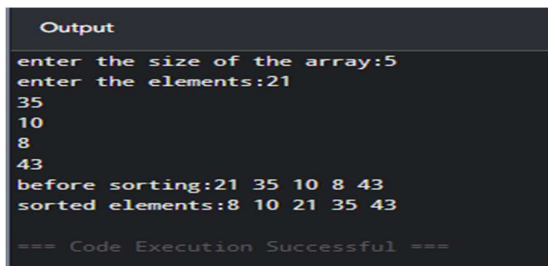
selection.logic();

cout << "\nsorted elements:";
selection.printelements();

return 0;
}

```

## OUTPUT:



```

Output
enter the size of the array:5
enter the elements:21
35
10
8
43
before sorting:21 35 10 8 43
sorted elements:8 10 21 35 43

=== Code Execution Successful ===

```

## TIME COMPLEXITY:

Time Complexity:

```
for (i=0; i<n-1; i++) {      → n-1
    int min=i;                → 1
    for (j=i+1; j<n; j++) {    → n
        if (arr[j] < arr[min]) { → 1
            min=j;            → 1
        }
    }
    if (min != i) {            → 1
        int temp = arr[i];     → 1
        arr[i] = arr[min];     → 1
        arr[min] = temp;       → 1
    }
}
```

$$\begin{aligned} T(n) &= (n-1)(1)(n) \times (1+1+1+\dots+1) \\ &= (n-1)(n) \\ &= n^2 - n \\ T(n) &= O(n^2) \end{aligned}$$

i) Best case: perform all comparison  
 $(n-1) + (n-2) + \dots + 1$   
 $T(n) = n(n-1)/2$   
 $= (n^2 - n)/2$   
 $T(n) = O(n^2)$

ii) Average case: check every pass  
 $T(n) = (n-1) + (n-2) + \dots + 1$   
 $= n(n-1)/2$   
 $T(n) = O(n^2)$

iii) worst case: all comparison  
 $T(n) = O(n^2)$

## 3.) INSERTION SORT:

### ALGORITHM:

3) Insertion Sort:

Algorithm:

Input: enter unsorted array elements  
Output: sorted elements

Steps: for  $i=1$  to  $i \leq n$  then

Step-2: temp =  $a[i]$  and  $j=i-1$  then

Step-3: while  $j > 0$  and  $a[j] > \text{temp}$

Step-4:  $a[j+1] = a[j]$  and  $j = j-1$

Step-5: stop while

Step-6:  $a[j+1] = \text{temp}$

Step-7: stop for

### PROGRAM:

```
#include <iostream>

using namespace std;

class insertionsort {

public:

    int arr[100], size, i, j, temp;

    void getinput() {

        cout << "enter the size of the array:";

        cin >> size;
```

```

    cout << "enter the elements:";

    for (i = 0; i < size; i++) {
        cin >> arr[i];
    }
}

void printelements() {
    for (i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
}

void logic() {
    for (i = 1; i < size; i++) {
        temp = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > temp) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = temp;
    }
}

};

int main() {
    insertionsort insertion;
    insertion.getinput();
    cout << "before sorting:";
    insertion.printelements();
    insertion.logic();
    cout << "sorted elements:";
    insertion.printelements();
    return 0;
}

```

## OUTPUT:

```
Output
enter the size of the array:5
enter the elements:9
7
2
5
10
before sorting:9 7 2 5 10 sorted elements:2 5 7 9 10

=== Code Execution Successful ===
```

## TIME COMPLEXITY:

Time complexity:

```
for (i=1; i<n; i++) {  
    temp = a[i];  
    j = i-1;  
    while (j >= 0 && arr[j] > temp) {  
        a[j+1] = a[j];  
        j--;  
    }  
    a[j+1] = temp;  
}
```

$\rightarrow n$   
 $\rightarrow 1$   
 $\rightarrow 1$   
 $\rightarrow n$   
 $\rightarrow 1$   
 $\rightarrow 1$   
 $\rightarrow 1$

$T(n) = n \times 1 \times 1 \times n \times 1 \times 1 \times 1$   
 $= n^2$   
 $T(n) = O(n^2)$

i) Best case: array is sorted  
one comparison  
 $n-1$   
 $T(n) = n-1$   
 $T(n) = O(n)$

ii) Average case:  
 $1+2+3+\dots+(n-1)$   
 $T(n) = n(n-1)/2$   
 $= (n^2 - n)/2$   
 $T(n) = O(n^2)$

iii) worst case:  
 $(n-1) + (n-2) + \dots + 1$   
 $T(n) = n(n-1)/2$   
 $T(n) = O(n^2)$

## 4.) QUICK SORT:

### ALGORITHM:

i) Quick Sort:  
Algorithm:  
Input: Enter unsorted array element  
Output: Sorted element  
Step 1: partition (arr, low, high) Then  
Step 2: Set pivot = arr[high]  
Set i = low-1  
Step 3: for j = low to high-1 Then  
Step 4: if arr[j] < pivot, Then  
i = i+1  
Swap arr[i] and arr[j]  
end if  
end for  
Step 5: Swap arr[i+1] and arr[high]  
Step 6: Return i+1  
Step 7: quicksort (arr, low, high) Then  
Step 8: if low < high Then do  
pivot index = partition (arr, low, high)  
quicksort (arr, low, pivotindex-1)  
quicksort (arr, Pivotindex+1, high)  
Step 9: end if

**PROGRAM:**

```
#include <stdio.h>

int partition(int arr[], int lb, int ub)
{
    int pivot = arr[lb];
    int start = lb;
    int end = ub;

    while (start < end)
    {
        while (arr[start] <= pivot)
        {
            start++;
        }
        while (arr[end] > pivot)
        {
            end--;
        }
        if (start < end)
        {
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
        }
    }

    int temp = arr[lb];
    arr[lb] = arr[end];
    arr[end] = temp;

    return end;
}

void quicksort(int arr[], int lb, int ub)
```

```
{
    if (lb < ub)
    {
        int index = partition(arr, lb, ub);
        quicksort(arr, lb, index - 1);
        quicksort(arr, index + 1, ub);
    }
}

int main()
{

    int arr[100], n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements: ");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    quicksort(arr, 0, n - 1);
    printf("Sorted array: ");
    for (int i = 0; i < n; i++)

    {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

## OUTPUT:

```
Output
Enter number of elements: 5
Enter array elements: 12
25
14
9
30
Sorted array: 9 12 14 25 30

=== Code Execution Successful ===
```

## TIME COMPLEXITY:

Time complexity:  
quicksort(arr, low, pivotindex-1);  $\rightarrow n/2$   
quicksort(arr, pivotindex+1, high);  $\rightarrow n/2$   
partition(arr, low, high);  $\rightarrow n$

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + n$$
$$= 2T\left(\frac{n}{2}\right) + n$$

using Master Method  
 $T(n) = aT(n/b) + f(n)$   
 $a=2, b=2, f(n)=n, p=0, k=1$   
 $n \log_a b = n \log_2 2 = n$   
 $\log_a b = k$   
Case-2:  $p > -1$   
 $T(n) = O(n \log^{p+1} n)$   
 $= O(n \log n)$

i) Best case (Pivot divides the array into two equal parts)  
 $n \rightarrow n/2 + n/2 \rightarrow n/4 + n/4 + n/4 + n/4 \rightarrow \dots \rightarrow 1$   
 $T(n) = n \times \log_2 n$   
 $T(n) = O(n \log n)$

ii) Average case: divides into balance parts  
 $n \rightarrow n/2 + n/2 \rightarrow n/4 + n/4 \dots$   
 $T(n) = n \times \log_2 n$   
 $T(n) = O(n \log n)$

iii) Worst case: (Pivot is always smallest or largest element)  
 $n \rightarrow (n-1) \rightarrow (n-2) \rightarrow (n-3) \rightarrow \dots \rightarrow 1$   
 $T(n) = (n-1) + (n-2) + \dots + 1$   
 $T(n) = n(n-1)/2$   
 $= (n^2 - n)/2$   
 $T(n) = O(n^2)$

## 5.) MERGE SORT

### ALGORITHM:

5) Merge Sort:  
Algorithm:  
Input: array elements  
Output: Sorted array.  
Step-1: merge(arr, low, mid, high) Then  
Step-2: Set  $i = \text{low}$ ,  $j = \text{mid} + 1$ ,  $k = \text{low}$   
Step-3: while  $i \leq \text{mid}$  and  $j \leq \text{high}$  Then  
Step-4: if  $\text{arr}[i] \leq \text{arr}[j]$  Then  
     $\text{temp}[k] = \text{arr}[i]$   
     $i = i + 1$   
else  
     $\text{temp}[k] = \text{arr}[j]$   
     $j = j + 1$   
end if

$k = k + 1$   
end while.  
Step-5: while  $j \leq \text{high}$  Then  
     $\text{temp}[k] = \text{arr}[j]$   
     $j = j + 1$   
     $k = k + 1$   
end while  
Step-6: while  $i \leq \text{mid}$  Then  
     $\text{temp}[k] = \text{arr}[i]$   
     $i = i + 1$   
     $k = k + 1$   
end while  
Step-7: for  $i = \text{low}$  to  $k = 0$  to  $i = \text{high}$  Then  
     $\text{arr}[i] = \text{temp}[k]$   
    (Step for)  
end for  
Step-8: mergeSort(arr, low, high) Then do  
Step-9: if  $\text{low} < \text{high}$  Then do  
     $\text{mid} = (\text{low} + \text{high}) / 2$   
    mergeSort(arr, low, mid)  
    mergeSort(arr, mid + 1, high)  
    merge(arr, low, mid, high)  
end if

### PROGRAM:

```
#include <stdio.h>
```

```
void merge(int arr[], int lb, int mid, int ub)
```

```
{  
    int i = lb;  
    int j = mid + 1;  
    int k = lb;  
    int b[100];
```

```
    while (i <= mid && j <= ub)
```

```
    {  
        if (arr[i] <= arr[j])  
        {  
            b[k] = arr[i];  
            i++;  
        }  
        else  
        {  
            b[k] = arr[j];  
            j++;  
        }  
        k++;  
    }
```

```
    while (i <= mid)
```

```
    {  
        b[k] = arr[i];  
        i++;  
        k++;  
    }
```

```
    while (j <= ub)
```

```

{
b[k] = arr[j];
j++;
k++;
}

for (i = lb; i <= ub; i++)
{
arr[i] = b[i];
}
}

void mergesort(int arr[], int lb, int ub)
{
if (lb < ub)
{
int mid = (lb + ub) / 2;

mergesort(arr, lb, mid);
mergesort(arr, mid + 1, ub);

merge(arr, lb, mid, ub);
}
}

int main()
{
int arr[100], n;

printf("Enter number of elements: ");
scanf("%d", &n);

printf("Enter array elements: ");
for (int i = 0; i < n; i++)
{
scanf("%d", &arr[i]);
}

mergesort(arr, 0, n - 1);

printf("Sorted array: ");
for (int i = 0; i < n; i++)
{
printf("%d ", arr[i]);
}

return 0;
}

```

### **OUTPUT:**

```

Output
Enter number of elements: 5
Enter array elements: 25
12
63
33
5
Sorted array: 5 12 25 33 63
=== Code Execution Successful ===

```

### TIME COMPLEXITY:

Time complexity:

$\text{mergesort}(arr, low, mid) \rightarrow n/2$   
 $\text{mergesort}(arr, mid+1, high) \rightarrow n/2$   
 $\text{merge}(arr, low, mid, high) \rightarrow n$   
 $T(n) = T(n/2) + T(n/2) + n$   
 $T(n) = 2T(n/2) + n$

Master Method:  $T(n) = aT(n/b) + f(n) \rightarrow a=2, b=2, f(n)=n, p=0, k=1$

$n \log_b a = n \log_2 2 = n$   
 $f(n) = n$   
 $n \log_2^2 n$  case 2:  $p > -1$

$T(n) = O(n \log^{p+1} n)$   
 $= O(n \log n)$

i) Best case: (divides array into two halves)

$n \rightarrow n/2 + n/2 \rightarrow n/4 + n/4 \dots \rightarrow 1$   
 $T(n) = n \times \log_2 n$   
 $T(n) = O(n \log n)$

ii) Average case:

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$   
 $T(n) = n \times \log_2 n$   
 $T(n) = O(n \log n)$

iii) Worst case: (Merge sort continues dividing the array into halves & merging all elements).

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \dots \rightarrow 1$   
 $T(n) = n \times \log_2 n$   
 $T(n) = O(n \log n)$